AMERICAN INTERNATIONAL UNIVERSITY–BANGLADESH (AIUB)

Dept. of Computer Science
Faculty of Science and Technology

CSC2210: OBJECT ORIENTED PROGRAMMING 2

Spring 2024-2025

Section: [*C*]

Group No: 01

Project Report On

*Project Name [**Tuition Solution**]*

Supervised By

Zahiduddin Ahmed

Submitted By:

Name	ID
1. Sajid Mohammad Talukdar	**23-53063-3**
2. Argho Dey	**23-55367-3**
3. Kazi Samia Mostofa	**23-54980-3**
4. Mohammad Saib	**23-54748-3**

CO2: Display and verify the mean of a real-life Project using the concepts of C# Graphical User Interface based environment with database integration to depict a desktop-based application.

Assessment Criteria	Not Attended/ Incorrect (0)	Inadequate (1-2)	Average (3)	Good (4)	Excellent (5)
Evaluation Criteria	**Evaluation Definition**				**Total =**
Requirement fulfillment	Properly demonstrate a real-life scenario-based project with proper functional requirement identification for the Object-Oriented Programming project development activities.				
Validation	Ensuring the ability of students' proper demonstration on validation forms in their system in terms of dealing with the data.				
Verification	Identifying if the students can verify the system data along with proper functional requirements in terms of data flow.				

Introduction

The Tuition Solution System is a form-based application designed to bridge the gap between students and teachers by offering an intuitive and secure platform for online tutoring requests and management. The system supports four distinct user roles: Admin, Student, Teacher, and General User (including unregistered or new users). Each role comes with specific permissions and responsibilities, ensuring a streamlined experience across all interactions.

Key features include:

Secure Login and Role Management using CPU ID verification and OTP authentication.

Role-based Dashboards tailored to teachers and students.

Tutoring Request Workflow including subject selection, scheduling, and status tracking.

Admin Controls for managing and moderating user access.

Advanced Search Functionality for students (by teacher qualification) and teachers (by subject).

Team Contribution Breakdown:

Member 1 – 35% (Lead Developer & Architect)
Main project Idea.

Designed the overall system architecture and database structure.

Implemented OTP-based password recovery.

Developed core Student and Teacher dashboard both frontend and backend.

Developed request creation (student side) and acceptance/rejection logic (teacher side).

Integrated CPU ID-based security features.

Overall backend development

Member 2 – 25% (Student & Teacher Module Developer)
Built student and teacher dashboards.

Implemented profile update features .

Developed secure login, role management in Login form

Added account deletion and allocation status management.

Member 3 – 20% (Admin & User Management Developer)
Developed admin login and user management panels.

Implemented account activation, suspension, and deletion functionalities.

Built interfaces to monitor and control system access for all users.

Developed Admin Dashboard and ints backend

Member 4 – 20% (Search & Notification Features Developer)
Implemented teacher search by qualification and student search by subject.

Developed feedback notification system (e.g., request sent, profile updated).

Enhanced user experience through UI-level improvements and form validations.

Developed User Regestation Logic and implement backend

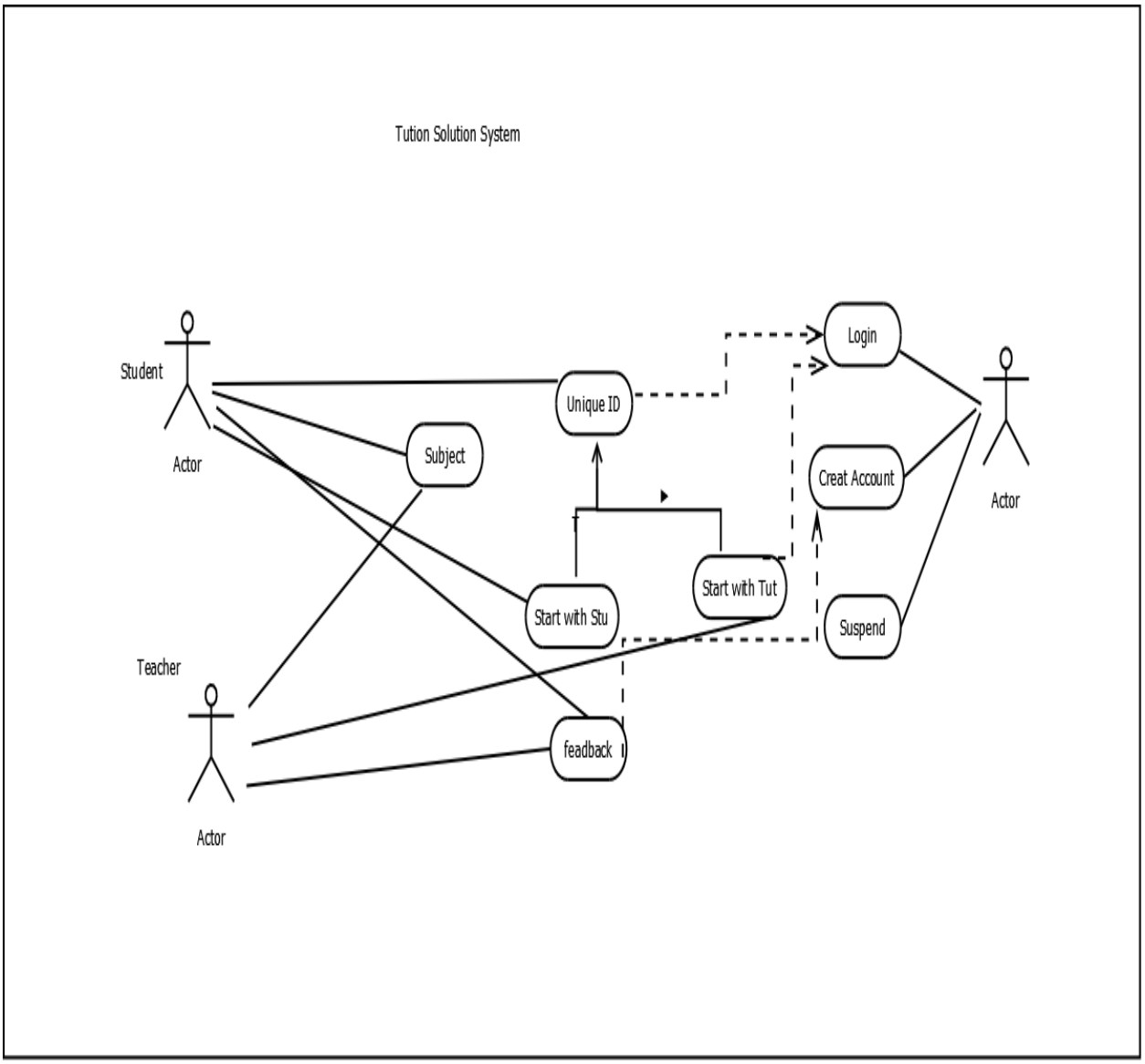
Example for User Stories

Use Case Name	Actor	User Story
Login Page	User	As a user, I want to log in from a single login page where I can enter my credentials.
Pre-Created Admin Account	Admin	As an admin, I want my account to be pre-created in the database by the application creator.
User Registration	Teacher/Student	As a teacher or student, I want to register and choose my role if I don't have an account.
Account Activation Requirement	User	As a user, I cannot log in unless my account is active (verified and allowed by the admin).
CPU ID Security	User	As a user, I want to use my CPU ID as a security check to prevent unauthorized logins.
Password Reset via OTP	User	As a user, I want to change my password using an OTP sent to my registered phone number.
Student Dashboard	Student	As a student, I want to see my personalized dashboard with my profile info and unique ID.
Student Profile Update	Student	As a student, I want to update my profile information.
Subject and Schedule Selection	Student	As a student, I want to select subjects, preferred time, and offered salary for tutoring.
Send Tutoring Request	Student	As a student, I want to send a tutoring request based on my selected preferences.

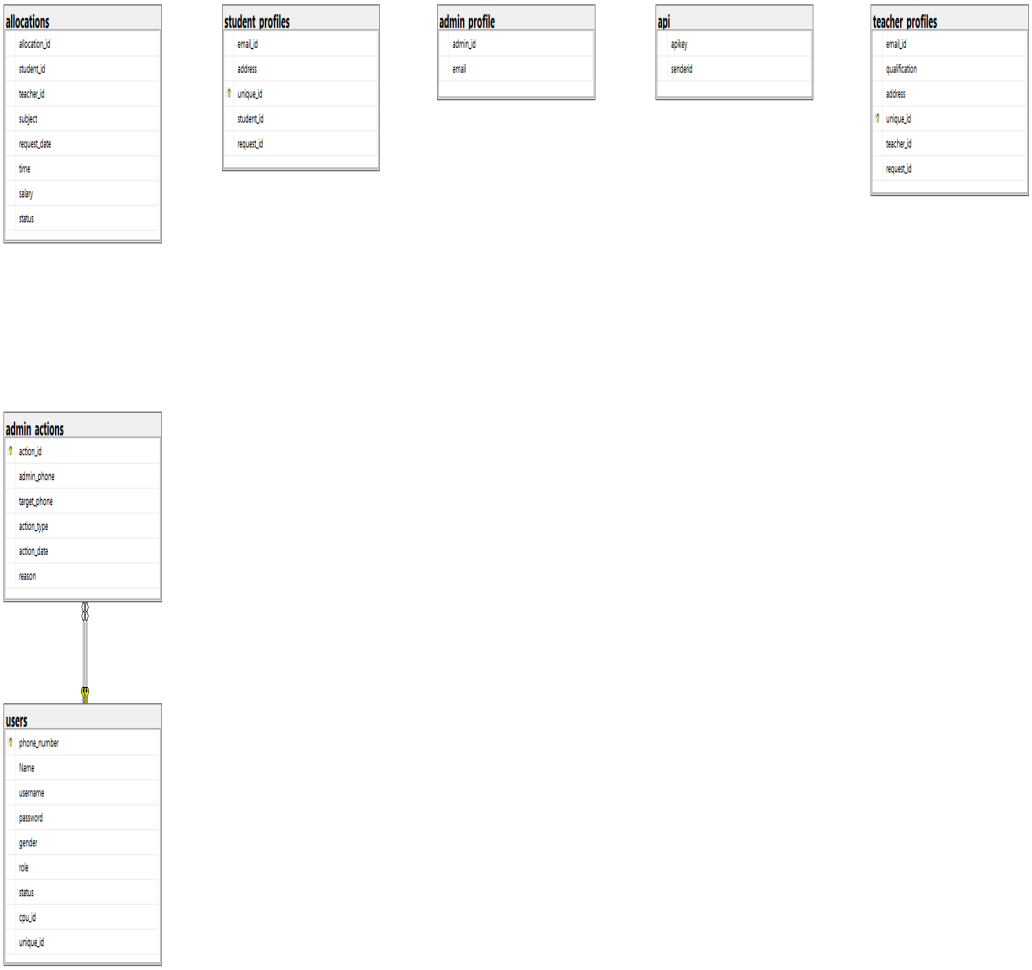
View Request Status	Student	As a student, I want to view the status of my requests.
Delete Student Account	Student	As a student, I want to delete my account if I wish.
Teacher Dashboard	Teacher	As a teacher, I want to see my personalized dashboard with profile info and unique ID.
Teacher Profile Update	Teacher	As a teacher, I want to update my profile information and qualifications.
View Student Requests	Teacher	As a teacher, I want to see pending student requests for tutoring.
Respond to Requests	Teacher	As a teacher, I want to accept or reject student requests.
Allocation Notification	Teacher	As a teacher, I want students to be notified when I accept a request and tutoring becomes active.
View Active Allocations	Teacher	As a teacher, I want to view my active tutoring allocations.
Delete Teacher Account	Teacher	As a teacher, I want to delete my account if I wish.
Admin Login	Admin	As an admin, I want to log in using special credentials from the database.
View All Users	Admin	As an admin, I want to view lists of all teachers and students with their statuses.
Manage Account Statuses	Admin	As an admin, I want to activate or suspend teacher or student accounts.
Delete Any User Account	Admin	As an admin, I want to delete any user account if

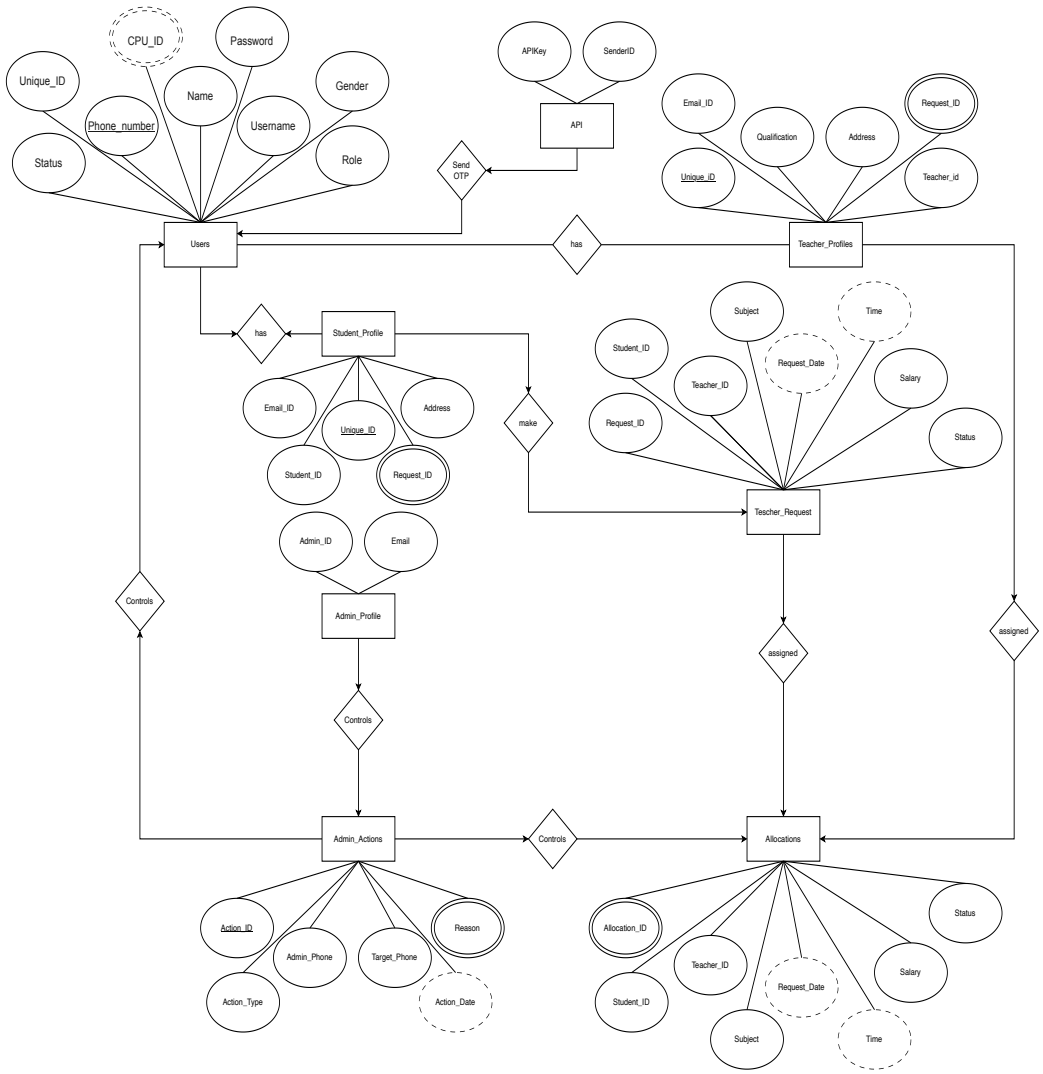
		necessary.
Security Verification	User	As a user, I want login verification based on my CPU ID.
OTP for Password Reset	User	As a user, I want to receive OTP on my registered phone for password changes.
Feedback Notifications	User	As a user, I want clear feedback messages for all operations.
Search Teachers by Qualification	Student	As a student, I want to search for teachers based on their qualifications.
Search Students by Subject	Teacher	As a teacher, I want to search for students based on the subjects they need help with.

UseCase Digram



Class Digram




CPU_ID
Password
Unique_ID
Name
Gender
Phone_number
Username
Status
Role
APIKey
SenderID
API
Send OTP
Email_ID
Qualification
Address
Request_ID
Unique_ID
Teacher_id
Users
has
Teacher_Profiles
has
Student_Profile
Subject
Time
Student_ID
Request_Date
Teacher_ID
Salary
Email_ID
Unique_ID
Address
make
Request_ID
Status
Student_ID
Request_ID
Tescher_Request
Admin_ID
Email
Controls
Admin_Profile
assigned
assigned
Controls
Admin_Actions
Controls
Allocations
Action_ID
Reason
Admin_Phone
Target_Phone
Action_Type
Action_Date
Allocation_ID
Status
Teacher_ID
Salary
Request_Date
Student_ID
Subject
Time

ER-Digram

Data Dictionary Document

Data Dictionary - users

Key	Name	Data Type	Length	Nullable
Primary	phone_number	nvarchar	20	No
	Name	nvarchar	30	No
	username	nvarchar	100	No
	password	nvarchar	100	No
	gender	nvarchar	10	No
	role	nvarchar	20	No
	status	nvarchar	20	No
	cpu_id	nvarchar	50	No
Primary	unique_id	nvarchar	50	No

Data Dictionary – student_profiles

Key	Name	Data Type	Length	Nullable
	email_id	nvarchar	20	Yes
	address	nvarchar	20	Yes
Primary	unique_id	nvarchar	20	No
	student_id	nvarchar	20	No
	request_id	nvarchar	20	No

Data Dictionary – admin_profiles

Key	Name	Data Type	Length	Nullable
Primary	admin_id	nchar	20	No
	email	nchar	20	No

Data Dictionary -teacher_requests

Key	Name	Data Type	Length	Nullable
Primary	request_id	nvarchar	30	No
	student_id	nvarchar	20	No
	teacher_id	nvarchar	20	No
	subject	nvarchar	20	No
	request_date	nvarchar	20	No
	time	nvarchar	20	No
	salary	nvarchar	20	No
	status	nvarchar	20	No

Data Dictionary - Allocation

Key	Name	Data Type	Length	Nullable
Primary	allocation_id	nvarchar	20	No
	student_id	nvarchar	20	No
	teacher_id	nvarchar	20	No
	subject	nvarchar	20	No
	request_date	nvarchar	20	No
	time	nvarchar	20	No
	salary	nvarchar	20	No
	status	nvarchar	20	No

Data Dictionary - teacher_profiles

Key	Name	Data Type	Length	Nullable
	email_id	nvarchar	20	Yes
	qualification	nvarchar	20	Yes
	address	nvarchar	20	Yes
Primary	unique_id	nvarchar	50	No
	teacher_id	nvarchar	20	No
	request_id	nvarchar	20	No

Data Dictionary - api

Key	Name	Data Type	Length	Nullable
Primary	apikey	nvarchar	20	No
	senderid	nvarchar	20	No

Normalization

Users **Controls** Admin_Actions

UNF

Controls(Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type)

1NF

Reason is a multi valued attribute

1. Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, Acteonid, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

2NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role

2. Acteonid, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

3NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role

2. Name, Username, Password

3. Acteonid, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

Table Creation

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username**

2. Name, Username, Password

3. Acteonid, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, **Phone_Number**

Send OTP

UNF

SendOtp(Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, SenderID, APIKey)

1NF

There is no multi valued attribute. Relation already in 1NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, SenderID, APIKey

2NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role

2. SenderID, APIKey

3NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role

2. Name, Username, Password

3. SenderID, APIKey

Table Creation

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username**

2. Name, Username, Password

3. SenderID, APIKey, **Phone_Number**

Users **Has** Teacher_Profiles

UNF

Has(Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id)

1NF

Request_ID is a multi valued attribute

1.Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id

2NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role

2. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id

3NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role

2. Name, Username, Password

3. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id

Table Creation

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username**

2. Name, Username, Password

3. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, **Phone_Number**

Teacher_Profiles **assigned** Allocations

UNF

assigned (Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject)

1NF

Allocation_ID and Request_ID are multi valued attributes

1.Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

2NF

1. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

3NF

There is no transitive dependency. Relation already in 3NF

1. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

Table Creation

1. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, **Allocation_ID**

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

Admin_Actions **Controls** Allocations

UNF

Controls(Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject)

1NF

Allocation_ID and Reason are multi valued attributes

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

2NF

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

3NF

There is no transitive dependency. Relation already in 3NF

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

Table Creation

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, **Allocation_ID**

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

Users **Has** Student_Profile

UNF

Has (Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, Unique_ID, Email_ID, Address, Student_ID, Request_ID)

1NF

Request_ID is a multi-valued attribute

1. Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role, Unique_ID, Email_ID, Address, Student_ID, Request_ID

2NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Name, Username, Password, Gender, Role

2. Unique_ID, Email_ID, Address, Student_ID, Request_ID

3NF

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role

2. Name, Username, Password

3. Unique_ID, Email_ID, Address, Student_ID, Request_ID

Table Creation

1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username, Unique_ID**

2. Name, Username, Password

3. Unique_ID, Email_ID, Address, Student_ID, Request_ID

Makes

UNF

Makes (Unique_ID, Request_ID Email_ID, Address, Student_ID, Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status)

1NF

Request_ID is a multi valued attribute

1. Unique_ID, Request_ID Email_ID, Address, Student_ID, Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status

2NF

1. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status

2. Unique_ID, Email_ID, Address, Student_ID, Request_ID

3NF

There is no transitive dependency. Relation already in 3NF

1. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status

2. Unique_ID, Email_ID, Address, Student_ID, Request_ID

Table Creation

1. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status,

2. Unique_ID, Request_ID, Email_ID, Address, Student_ID, **Request_ID**

Admin_Actions **Controls** Admin_Profile

UNF

Controls(Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, Admin_ID, Email)

1NF

Allocation_ID and Reason are multi valued attribute

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, Admin_ID, Email

2NF

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

2. Admin_ID, Email

3NF

There is no transitive dependency. Relation already in 3NF

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

2. Admin_ID, Email

Table Creation

1. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type

2. Admin_ID, Email, **Action_ID**

Tescher_Request **assigned** Allocations

UNF

assigned (Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status, Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject)

1NF

Allocation_ID is a multi-valued attribute

1.Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status, Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

2NF

1. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

3NF

There is no transitive dependency. Relation already in 3NF

1. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

Table Creation

1. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status, **Allocation_ID**

2. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

Temporary Tables

~~1. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username**~~

2. Name, Username, Password

3. Acteonid, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, **Phone_Number**

~~4. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username**~~

~~5. Name, Username, Password~~

6. SenderID, APIKey, **Phone_Number**

~~7. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username**~~

~~8. Name, Username, Password~~

9. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, **Phone_Number**

10. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, **Allocation_ID**

11. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

12. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, **Allocation_ID**

~~13. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject~~

14. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username, Unique_ID**

~~15. Name, Username, Password~~

~~16. Unique_ID, Email_ID, Address, Student_ID, Request_ID~~

~~17. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status,~~

18. Unique_ID, Request_ID, Email_ID, Address, Student_ID, **Request_ID**

~~19. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type~~

20. Admin_ID, Email, **Action_ID**

21. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status, **Allocation_ID**

~~22. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject~~

Final Tables

1. Name, Username, Password

2. Acteonid, Admin_Phone, Target_Phone, Action_Date, Reason, Action_Type, **Phone_Number**

3. SenderID, APIKey, **Phone_Number**

4. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, **Phone_Number**

5. Unique_iD, Email_ID, Qualification, Address, Request_ID, Teacher_id, **Allocation_ID**

6. Allocation_ID, Student_ID, Teacher_ID, Request_Date, Time, Salary, Status, Subject

7. Action_ID, Admin_Phone, Target_Phone, Action_Date, Reason1, Reason2 Action_Type, **Allocation_ID**

8. Phone_Number, Status, Unique_Id, Cpu_ID, Gender, Role, **Username, Unique_ID**

9. Unique_ID, Request_ID1, Request_ID2, Email_ID, Address, Student_ID, **Request_ID**

10. Admin_ID, Email, **Action_ID**

11. Request_ID, Student_ID, Teacher_ID, Subject, Request_Date, Time, Salary, Status, **Allocation_ID**

SQL Queries Reference

Validate User Credentials (Retrieve User)

Table: users

```
SELECT * FROM users WHERE username = @username AND password =
@password;
```

Check User Existence by Credentials (Count)

Table: users

```
SELECT COUNT(*) FROM users WHERE username = @username AND password =
@password;
```

Update CPU ID by Username

Table: users

```
UPDATE users SET cpu_id = @cpu_id WHERE username = @username;
```

Update CPU ID by Unique ID

Table: users

```
UPDATE users SET cpu_id = @cpu_id WHERE unique_id = @unique_id;
```

Check Existing Username Count

Table: users

```
SELECT COUNT(*) FROM users WHERE username = @username;
```

Insert New User

Table: users

```
INSERT INTO users (phone_number, Name, username, password, gender,
role, status, cpu_id, unique_id)
VALUES (@phone_number, @Name, @username, @password, @gender, @role,
'SUSPENDED', @cpu_id, @unique_id);
```

Insert Student Profile

Table: student_profiles

```
INSERT INTO student_profiles (unique_id, student_id) VALUES
(@unique_id, @student_id);
```

Insert Teacher Profile

Table: teacher_profiles

```
INSERT INTO teacher_profiles (unique_id, teacher_id) VALUES
(@unique_id, @teacher_id);
```

Get Users by Status and Role(s)

Table: users

```
SELECT * FROM users WHERE status = @status AND (role = @role1 OR role =
@role2);
```

Delete Allocations by Student or Teacher ID

Table: allocations

```
DELETE FROM allocations WHERE student_id = @unique_id OR teacher_id =
@unique_id;
```

Delete Student Profile by Unique ID

Table: student_profiles

```
DELETE FROM student_profiles WHERE unique_id = @unique_id;
```

Delete Teacher Profile by Unique ID

Table: teacher_profiles

```
DELETE FROM teacher_profiles WHERE unique_id = @unique_id;
```

Delete User by Unique ID

Table: users

```
DELETE FROM users WHERE unique_id = @unique_id;
```

Load Active Teachers with Profile

Table: users, teacher_profiles

```
SELECT
    u.Name AS [Teacher Name],
    u.phone_number AS [Phone Number],
    tp.qualification AS Qualification,
    tp.address AS Address,
    tp.teacher_id AS [Teacher ID]
FROM
    users u
JOIN
    teacher_profiles tp ON u.unique_id = tp.unique_id
WHERE
    u.status = 'ACTIVE' AND u.role = 'TEACHER';
```

Load Student Info by Unique ID

Table: student_profiles

```
SELECT
    email_id,
    address
FROM
    student_profiles
WHERE
    unique_id = @UniqueID;
```

Insert Teacher Request

Table: teacher_requests

```
INSERT INTO teacher_requests (
    request_id,
    student_id,
    teacher_id,
    subject,
    request_date,
    time,
    salary,
    status
) VALUES (
    @RequestID,
    @StudentID,
    @TeacherID,
    @Subject,
```

```
    @RequestDate,
    @Time,
    @Salary,
    'PENDING'
);
```

Search Teachers by Qualification

Table: users, teacher_profiles

```
SELECT
    u.Name AS [Teacher Name],
    u.phone_number AS [Phone Number],
    tp.qualification AS Qualification,
    tp.address AS Address,
    tp.teacher_id AS [Teacher ID]
FROM
    users u
JOIN
    teacher_profiles tp ON u.unique_id = tp.unique_id
WHERE
    tp.qualification = @Qualification;
```

Update Student Profile (Email and Address)

Table: student_profiles

```
UPDATE
    student_profiles
SET
    email_id = @Email,
    address = @Address
WHERE
    unique_id = @UniqueID;
```

Load Active Allocations for Student

Table: users, teacher_profiles, allocations

```
SELECT
    u.Name AS [Teacher Name],
    u.role AS [Role],
    al.subject AS Subject,
    u.phone_number AS [Phone Number],
    al.time AS [Time],
    al.salary AS [Salary],
    tp.qualification AS Qualification,
```

```
    tp.address AS Address,
    al.allocation_id
FROM
    users u
JOIN
    teacher_profiles tp ON u.unique_id = tp.teacher_id
JOIN
    allocations al ON tp.teacher_id = al.teacher_id
WHERE
    al.status = 'ACTIVE' AND al.student_id = @StudentID;
```

Delete Student Data Cascade

Table: allocations, student_profiles, users

```
DELETE FROM allocations WHERE student_id = @StudentID;
DELETE FROM student_profiles WHERE student_id = @StudentID;
DELETE FROM users WHERE unique_id = @StudentID;
```

Update Teacher Profile Info

Table: teacher_profiles

```
UPDATE teacher_profiles
SET email_id = @Email, address = @Address, qualification =
@Qualification
WHERE unique_id = @UniqueID;
```

Update Teacher's Basic Info

Table: users

```
UPDATE users
SET phone_number = @Phone, Name = @Name
WHERE unique_id = @UniqueID;
```

Load Active Students for Teacher

Table: users, student_profiles, allocations

```
SELECT
    u.Name AS [Student Name],
    u.role AS [Role],
    al.subject AS Subject,
    u.phone_number AS [Phone Number],
    al.time AS [Time],
    al.salary AS [Salary],
```

```
    sp.address AS Address,
    al.allocation_id
FROM
    users u
    JOIN student_profiles sp ON u.unique_id = sp.student_id
    JOIN allocations al ON sp.student_id = al.student_id
WHERE al.status = 'ACTIVE' AND al.teacher_id = @UniqueID;
```

Delete Teacher Data Cascade

Table: allocations, teacher_profiles, users

```
DELETE FROM allocations WHERE teacher_id = @UniqueID;
DELETE FROM teacher_profiles WHERE teacher_id = @UniqueID;
DELETE FROM users WHERE unique_id = @UniqueID;
```

Load Teacher Requests by Subject

Table: users, student_profiles, teacher_requests

```
SELECT
    u.Name AS [Student Name],
    al.subject AS Subject,
    u.phone_number AS [Phone Number],
    al.time AS [Time],
    al.salary AS [Salary],
    sp.address AS Address,
    sp.student_id AS [Student ID],
    al.request_id AS [Request ID]
FROM
    users u
    JOIN student_profiles sp ON u.unique_id = sp.student_id
    JOIN teacher_requests al ON al.student_id = sp.student_id
WHERE al.teacher_id = @UniqueID AND al.subject = @Subject;
```

Get Teacher Profile Details

Table: teacher_profiles, users

```
SELECT
    tp.email_id,
    tp.address,
    tp.qualification
FROM teacher_profiles tp
JOIN users u ON u.unique_id = tp.unique_id
WHERE u.unique_id = @UniqueID;
```

Insert Allocation from Teacher Request

Table: allocations

```
INSERT INTO allocations (allocation_id, student_id, teacher_id,
subject, request_date, time, salary, status)
SELECT request_id, student_id, teacher_id, subject, request_date, time,
salary, status
FROM teacher_requests
WHERE request_id = @RequestID AND teacher_id = @UniqueID;
```

Delete Teacher Request After Allocation

Table: teacher_requests

```
DELETE FROM teacher_requests WHERE request_id = @RequestID AND
teacher_id = @UniqueID;
```

Update Allocation Status to ACTIVE

Table: allocations

```
UPDATE allocations SET status = 'ACTIVE' WHERE allocation_id =
@RequestID AND teacher_id = @UniqueID;
```

Delete Teacher Request

Table: teacher_requests

```
DELETE FROM teacher_requests WHERE teacher_id = @UniqueID AND
request_id = @RequestID;
```

Load Pending Teacher Requests

Table: users, student_profiles, teacher_requests

```
SELECT
    u.Name AS [Student Name],
    u.phone_number AS [Phone Number],
    al.subject AS Subject,
    al.time AS [Time],
    al.salary AS [Salary],
    sp.address AS Address,
    sp.student_id AS [Student ID],
    al.request_id AS [Request ID]
FROM users u
```

```
JOIN student_profiles sp ON u.unique_id = sp.unique_id
JOIN teacher_requests al ON al.student_id = sp.student_id
WHERE al.status = 'PENDING' AND u.role = 'STUDENT' AND al.teacher_id =
@UniqueID;
```

Select Suspended Users (Optional Filter by Unique ID)

Table: users, student_profiles, teacher_profiles

```
SELECT
    u.Name AS name,
    u.phone_number,
    u.role AS role,
    u.status AS status,
    u.unique_id,
    COALESCE(sp.email_id, tp.email_id) AS email,
    COALESCE(sp.address, tp.address) AS address
FROM
    users u
LEFT JOIN
    student_profiles sp ON u.unique_id = sp.student_id AND u.role =
'student'
LEFT JOIN
    teacher_profiles tp ON u.unique_id = tp.teacher_id AND u.role =
'teacher'
WHERE
    u.status = 'suspended'
    AND (@unique_id IS NULL OR sp.student_id = @unique_id OR
tp.teacher_id = @unique_id);
```

Select Active Students

Table: users, student_profiles

```
SELECT
    u.Name AS Name,
    u.phone_number AS Phone,
    u.role AS Role,
    sp.address AS Address,
    sp.email_id AS Email
FROM
    users u
JOIN
    student_profiles sp ON u.unique_id = sp.student_id
WHERE
    u.status = 'ACTIVE'
    AND u.role = 'STUDENT';
```

Select Active Teachers

Table: users, teacher_profiles

```
SELECT
    u.Name AS Name,
    u.phone_number,
    u.role AS Role,
    sp.address AS Address,
    sp.email_id AS Email
FROM
    users u
JOIN
    teacher_profiles sp ON u.unique_id = sp.teacher_id
WHERE
    u.status = 'ACTIVE'
    AND u.role = 'TEACHER';
```

Update User Status from SUSPENDED to ACTIVE

Table: users

```
UPDATE users
SET status = 'ACTIVE'
WHERE status = 'SUSPENDED' AND unique_id = @unique_id;
```

Delete User by Unique ID

Table: users

```
DELETE FROM users WHERE unique_id = @unique_id;
```

Select Admin Profile

Table: admin_profile

```
SELECT * FROM admin_profile;
```

Update Admin Email

Table: admin_profile

```
UPDATE admin_profile SET email = @admin_email;
```

Update Admin Phone Number and Name

Table: users

```
UPDATE users
SET phone_number = @phone_number, Name = @name
WHERE unique_id = @admin_id;
```

Delete Allocation by Allocation ID

Table: allocations

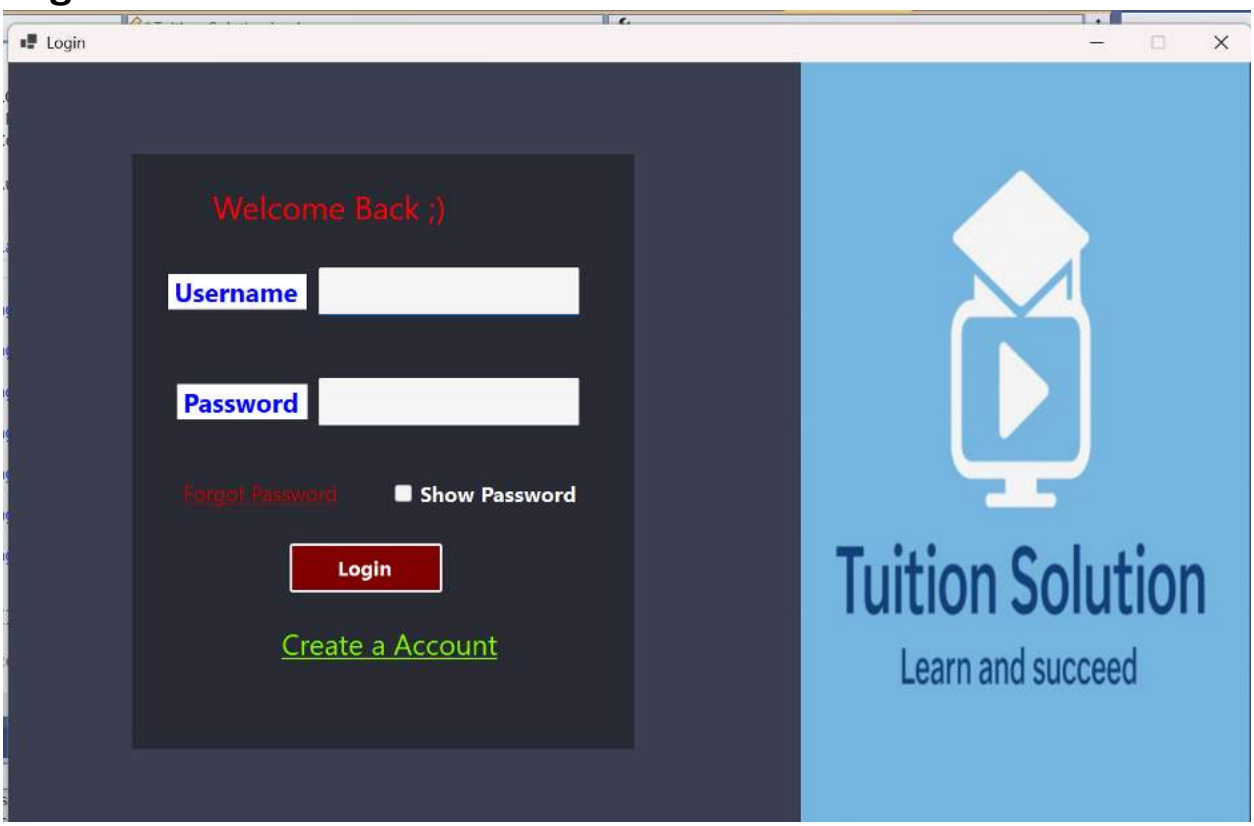
```
DELETE FROM allocations WHERE allocation_id = @allocation_id;
```

Update User Password

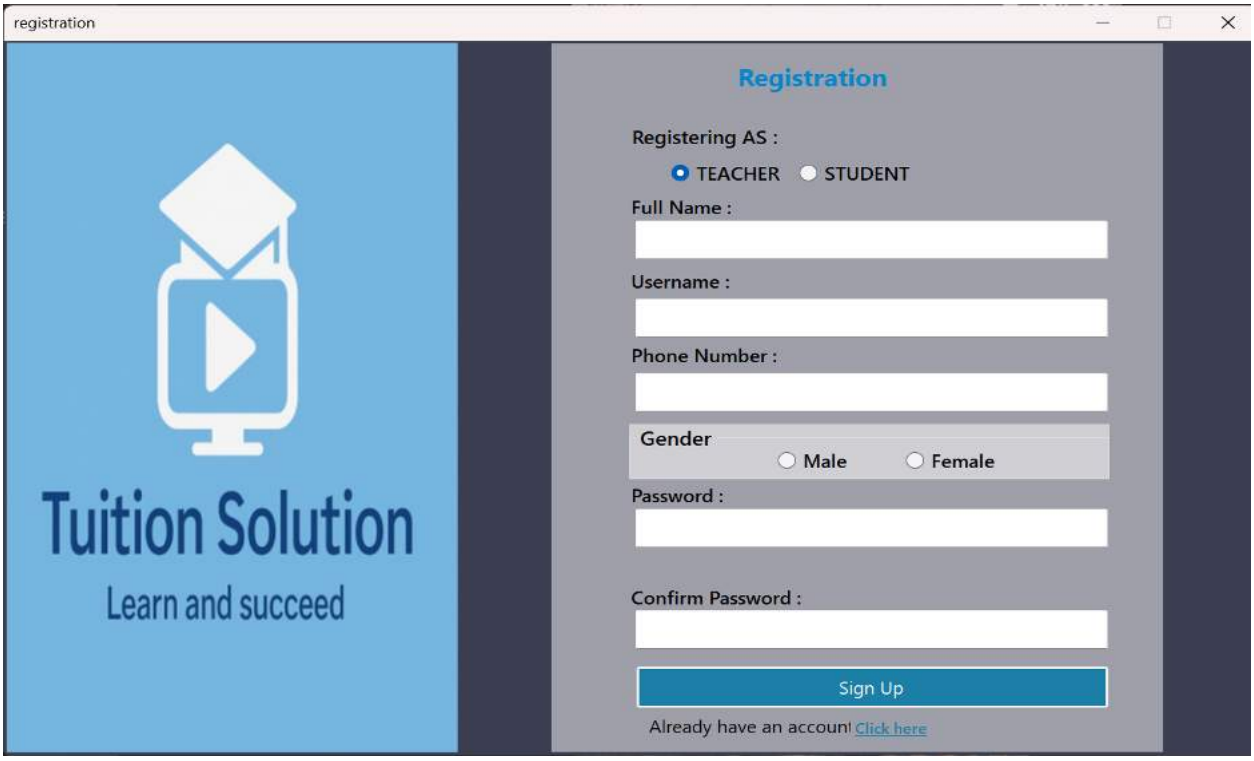
Table: users

```
UPDATE users SET password = @password WHERE unique_id = @unique_id;
```

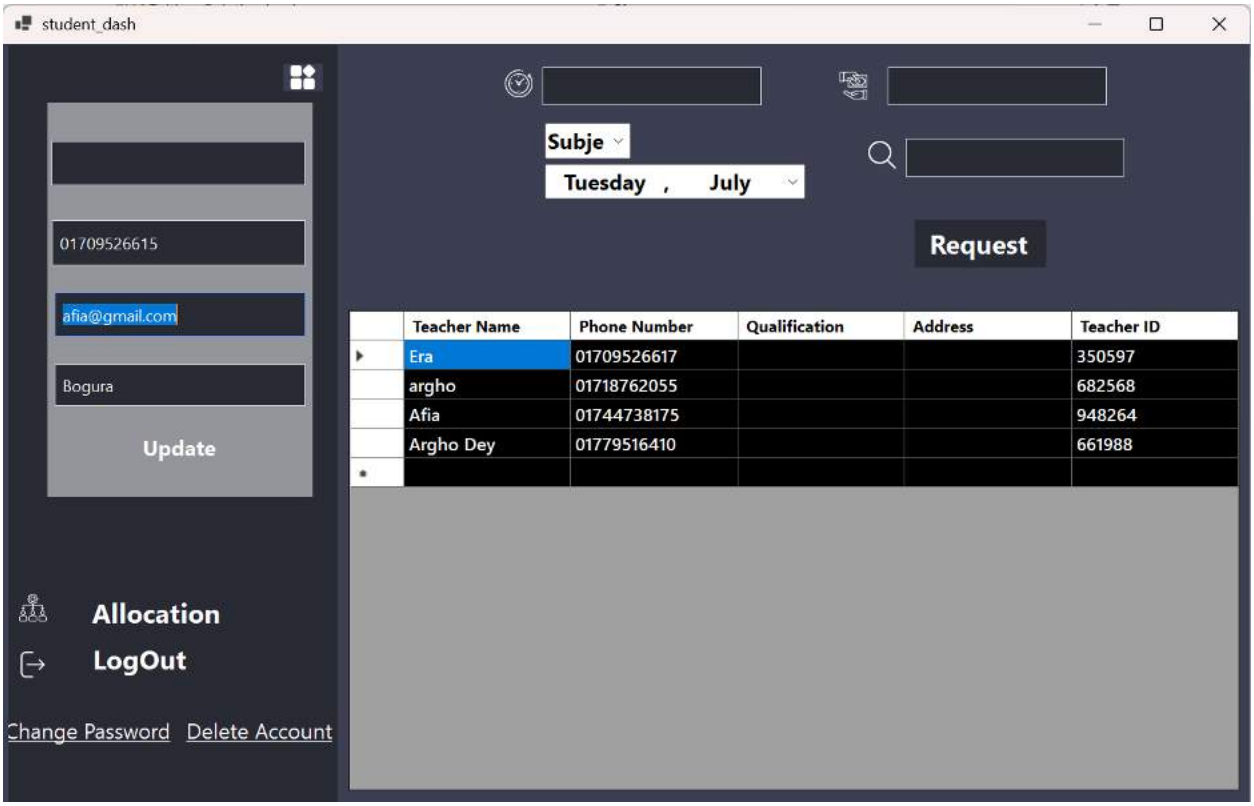
Login Frame



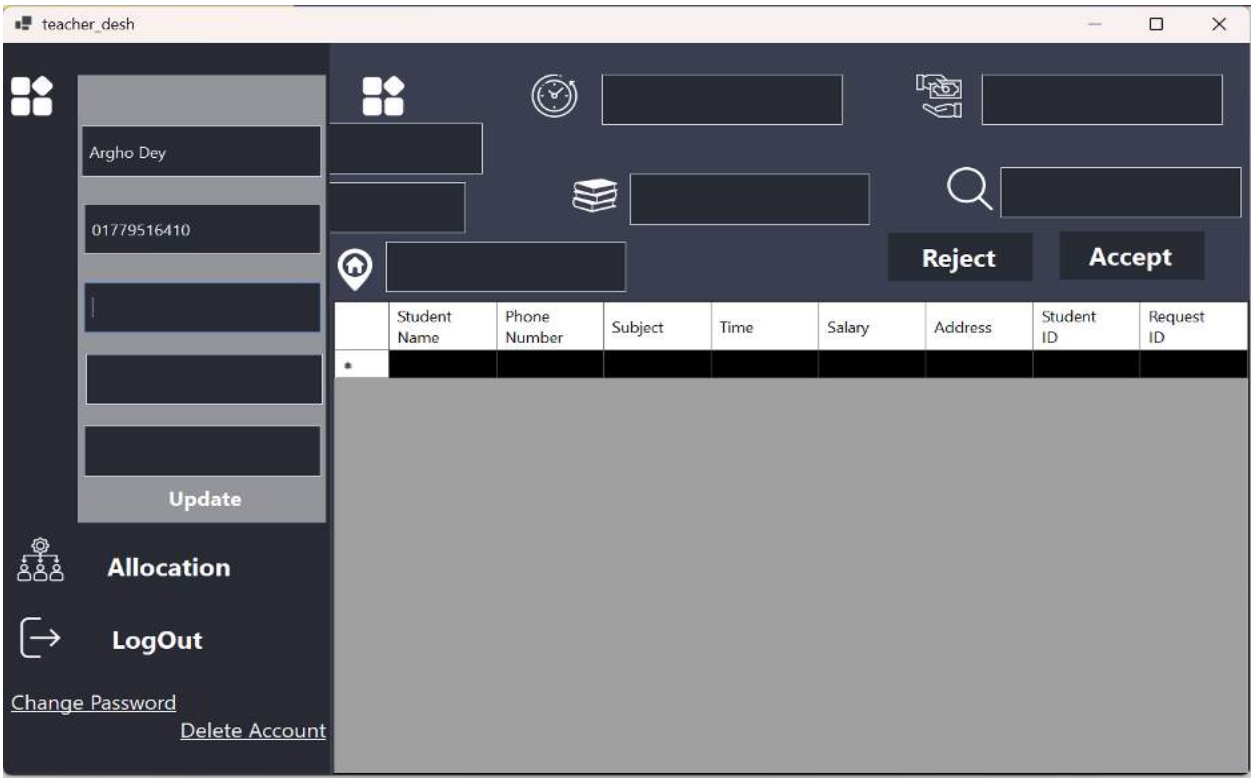
Registration Frame



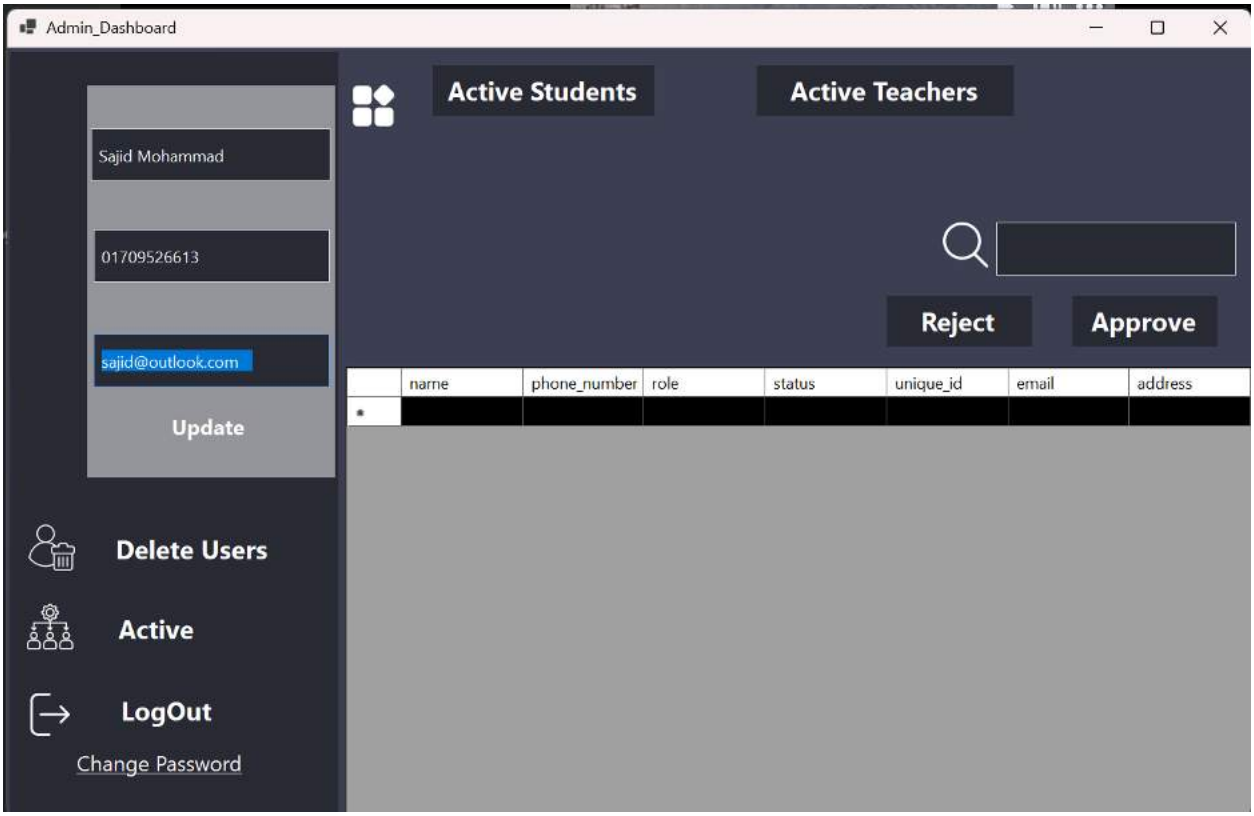
Student Dashboard Frame



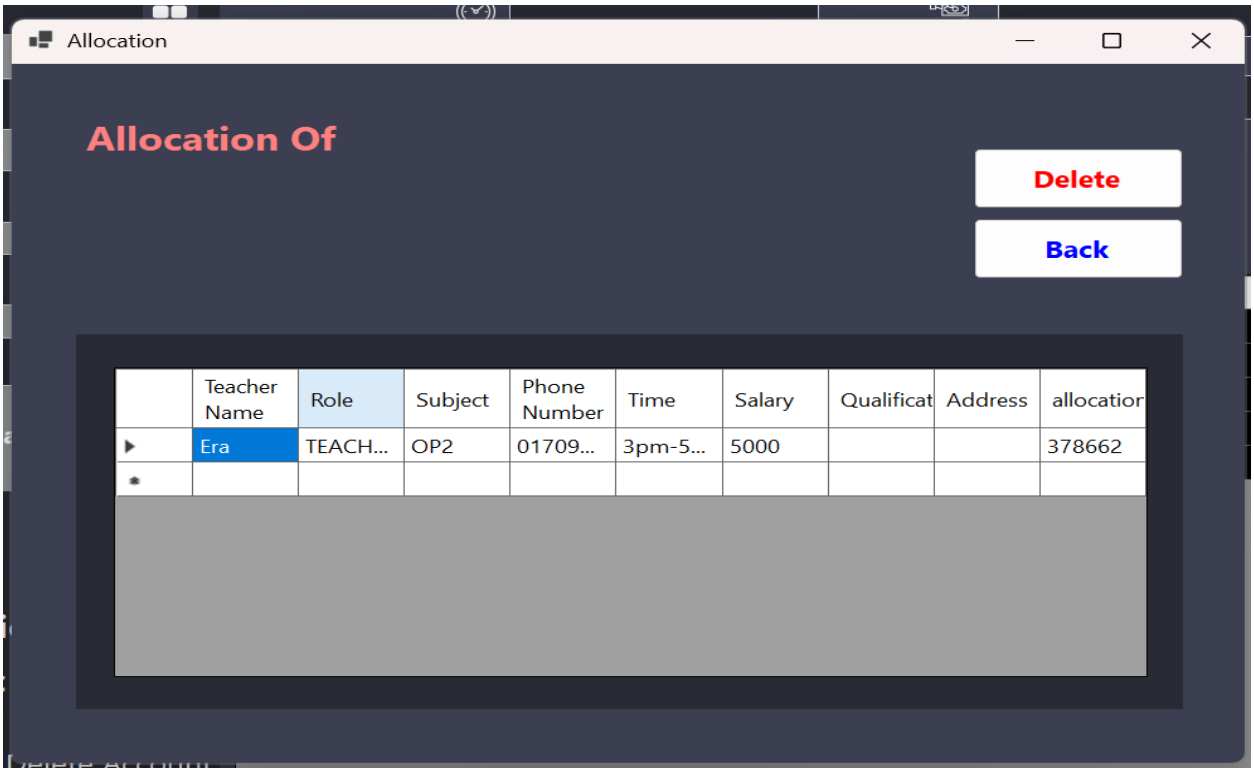
Teacher Dashboard Frame



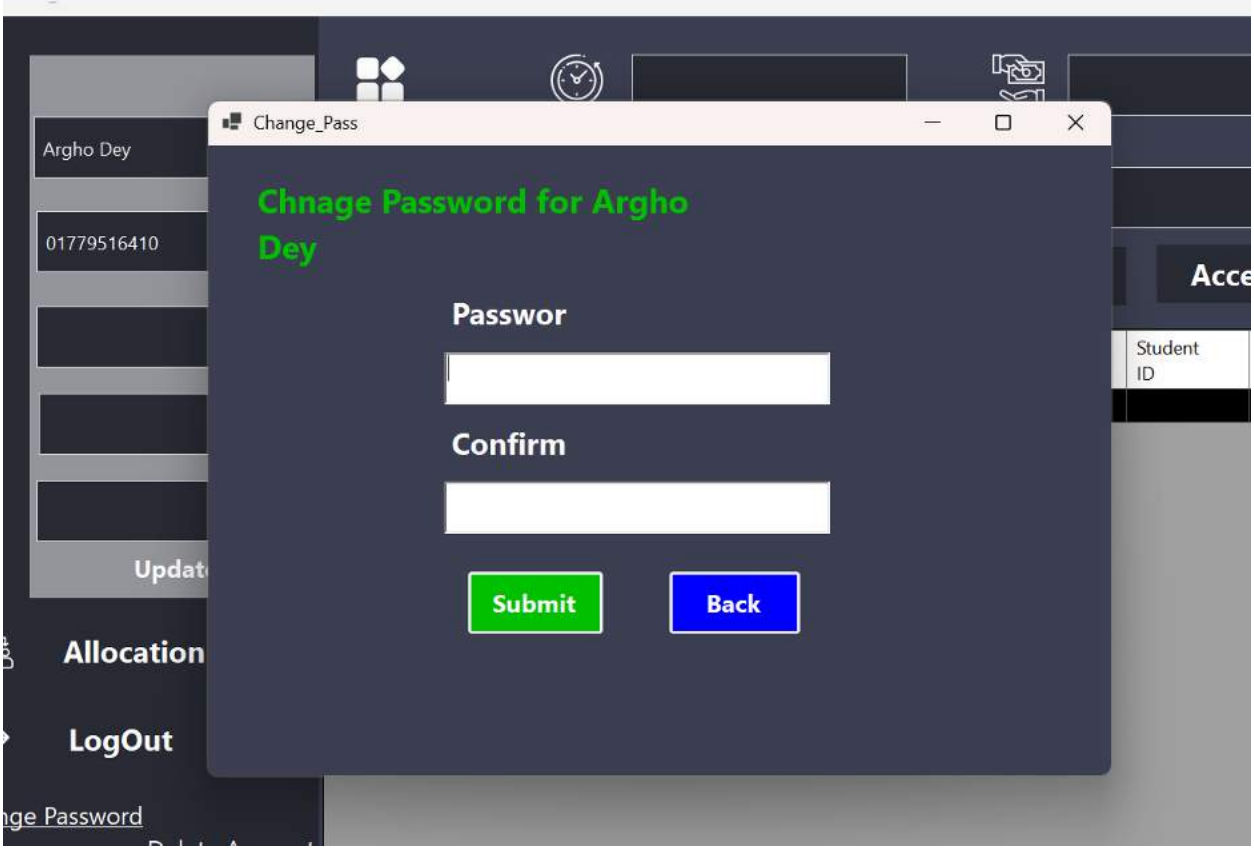
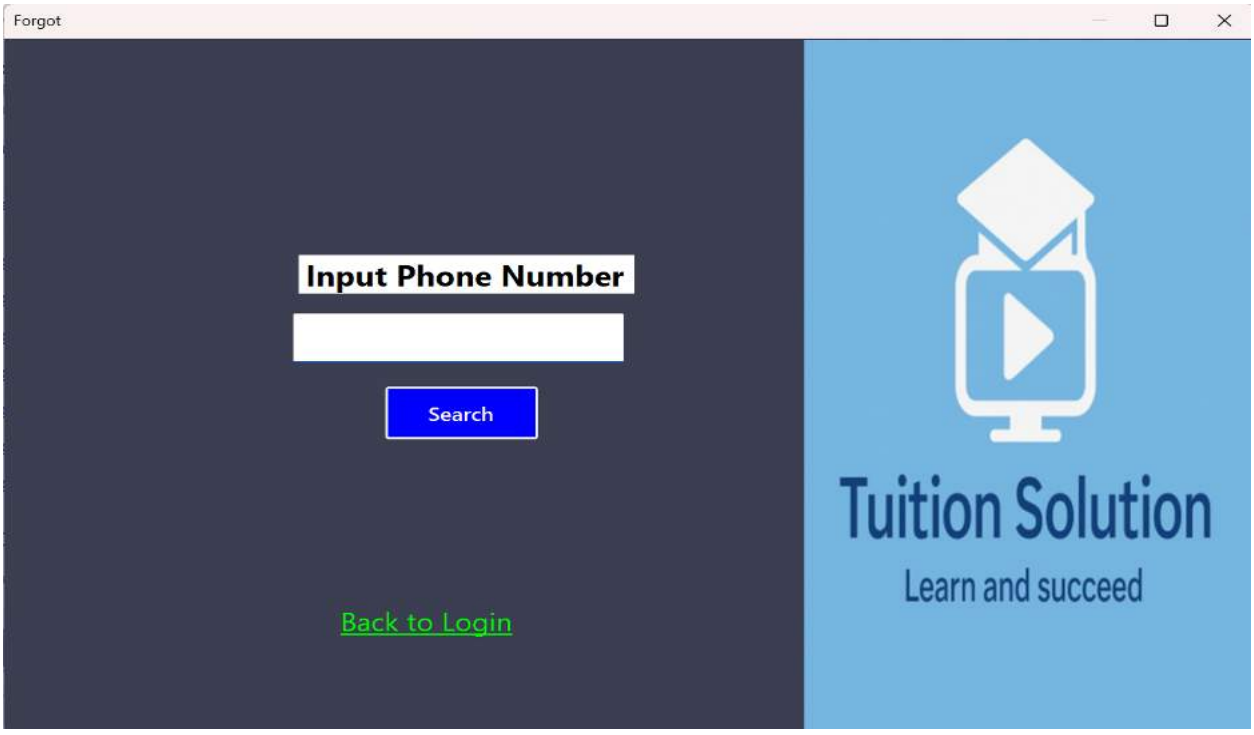
Admin Dashboard Frame



Allocations Frame



Change Password Frame



Forgot Password Frame